

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Algorithm Design Challenge

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 90 Mins

No. of Questions: 5

Annexure A

Algorithm Design Challenge

Code of Conduct:

I M . Sanjay Krishna(192525421)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

QUESTION 1 – Greedy Best-First Search

Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the Romania Map Problem. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable.

1. Problem Statement

Find a path from Arad to Bucharest using Greedy Best-First Search. The algorithm selects the node that appears closest to the goal using $h(n)$.

2. Problem Analysis

- Initial state: Arad
- Goal state: Bucharest
- State: Current city
- Action: Travel to a connected city
- Heuristic: Straight-line distance to Bucharest

3. AI Technique Selection

Greedy Best-First Search.

4. Justification of Algorithm

It is suitable because it uses heuristic information to move toward the goal quickly. It is simple and often reduces unnecessary exploration

6. Algorithm / Pseudocode

```
Greedy_Best_First_Search (start, goal)
1. Put start in OPEN.
2. Create CLOSED.
3. While OPEN is not empty:
  a. Select node n with the smallest h(n).
  b. Remove n from OPEN.
  c. If n is the goal, return the path.
  d. Add n to CLOSED.
  e. Generate successors.
  f. Add unvisited successors to OPEN.
4. Return failure.
```

7. Working / Execution Trace

Illustrative standard Romania-map trace: Arad $\rightarrow$ Sibiu $\rightarrow$ Fagaras $\rightarrow$ Bucharest.
The path shown is an example because the exact graph is not present in the uploaded assessment.

8. Complexity Analysis

Time: $O(b^m)$. Space: $O(b^m)$.

9. Advantages

- Simple; uses heuristic information; often reaches the goal quickly.

10. Limitations

- Does not guarantee the shortest path and may be misled by a poor heuristic.

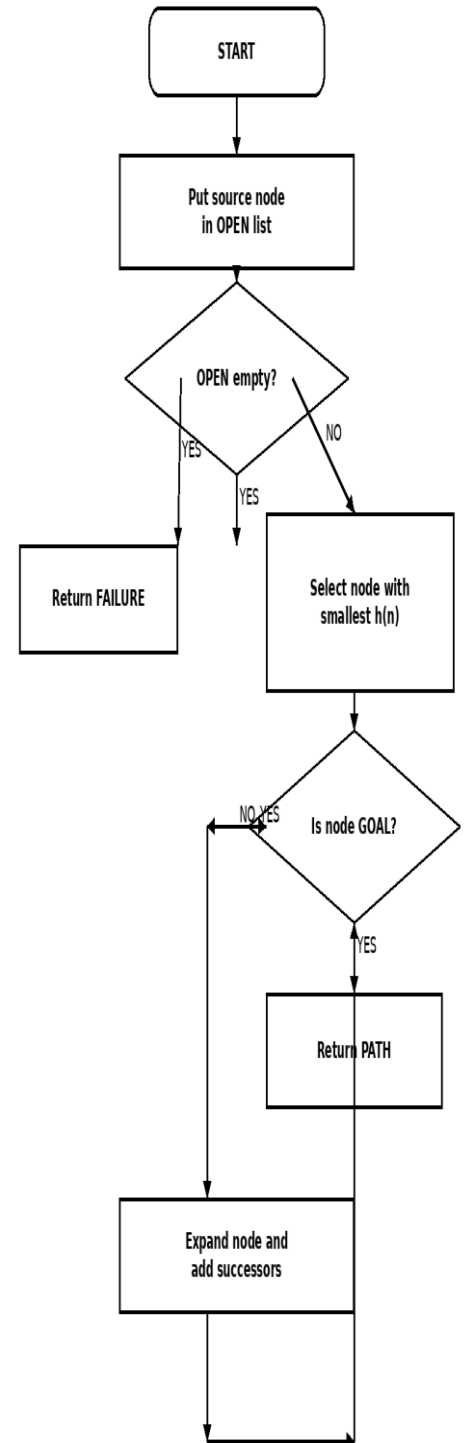
11. Conclusion

Greedy Best-First Search is useful when a quick goal-directed solution is required, but it does not guarantee an optimal path.

Heuristic function: $f(n)=h(n)$. Example h values: Arad=366, Sibiu=253, Fagaras=176, Rimnicu Vilcea=193, Pitesti=100, Bucharest=0.

5. Flowchart / Algorithm Diagram

Greedy Best-First Search Flowchart



QUESTION 2 – A* Search

A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A* Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A* guarantees the optimal solution.

1. Problem Statement

A* Search finds a path by combining the actual cost already travelled and the estimated remaining cost.

2. Problem Analysis

- Initial state: Warehouse
- Goal: Customer
- State: Current location
- Action: Move through connected roads
- $g(n)$: Actual cost from start
- $h(n)$: Estimated remaining cost

3. AI Technique Selection

A* Search.

4. Justification of Algorithm

A* considers both the path cost already travelled and the estimated cost to the goal. With an admissible heuristic, it can guarantee an optimal solution.

5. Flowchart / Algorithm Diagram

6. Algorithm / Pseudocode

```
A_Star(start, goal)
1. Put start in OPEN.
2. Set  $g(\text{start})=0$ .
3. Calculate  $f(\text{start})=g(\text{start})+h(\text{start})$ .
4. While OPEN is not empty:
  a. Select node with minimum  $f(n)$ .
  b. If node is goal, return optimal path.
  c. Expand the node.
  d. Calculate  $g(n)$ ,  $h(n)$ , and  $f(n)$  for successors.
  e. Add or update successors in OPEN.
5. Return failure.
```

7. Working / Execution Trace

Illustrative values: A: $g=0$, $h=7$, $f=7$; B: $g=2$, $h=6$, $f=8$; D: $g=7$, $h=2$, $f=9$. A* chooses the available node with the smallest $f(n)$.

8. Complexity Analysis

Worst-case time: $O(b^d)$. Worst-case space: $O(b^d)$.

9. Advantages

- Can find an optimal path with an admissible heuristic; combines actual and estimated cost.

10. Limitations

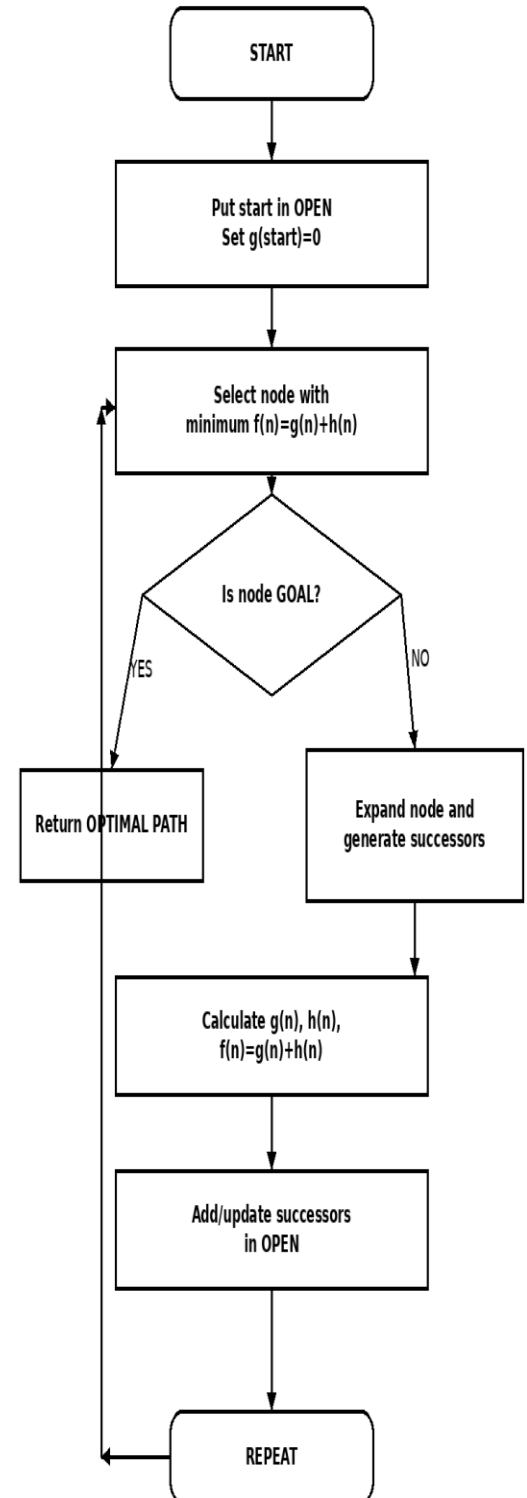
- Can require large memory; performance depends on the heuristic.

11. Conclusion

A* is suitable for the delivery robot because it balances the cost already travelled and the estimated cost to the destination.

Formula: $f(n)=g(n)+h(n)$. Admissibility condition: $h(n)$ must not overestimate the true remaining cost.

A* Search Flowchart



QUESTION 3 – Examination Timetable Using CSP

A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a CSP by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve efficiency.

1. Problem Statement

Schedule university subjects into examination slots so that subjects having conflicts are not scheduled at the same time.

2. Problem Analysis

- Variables: Mathematics, Physics, DBMS, AI, Computer Networks
- Domains: Monday Morning, Monday Afternoon, Tuesday Morning, Tuesday Afternoon
- Constraints: Conflicting subjects must have different time slots

3. AI Technique Selection

Constraint Satisfaction Problem (CSP) with MRV and Forward Checking.

4. Justification of Algorithm

CSP naturally represents scheduling because subjects are variables, examination slots are domains, and conflicts are constraints. MRV chooses the most restricted variable first, while Forward Checking removes invalid values early.

6. Algorithm / Pseudocode

CSP_Backtracking(assignment)

1. If all variables are assigned, return assignment.
2. Select an unassigned variable using MRV.
3. For each possible value:
 - a. Check constraints.
 - b. Assign the value.
 - c. Apply Forward Checking.
 - d. If consistent, continue recursively.
 - e. If failure occurs, remove the assignment.
4. Return failure if no value works.

7. Working / Execution Trace

Example: If Physics is assigned Monday Morning, that slot is removed from the domains of all subjects that conflict with Physics. If a domain becomes empty, the algorithm backtracks.

8. Complexity Analysis

Worst-case time: $O(d^n)$, where n is the number of variables and d is the number of possible values.

9. Advantages

- Natural representation for timetable problems; MRV and Forward Checking reduce unnecessary search.

10. Limitations

- Large CSPs can still require substantial computation.

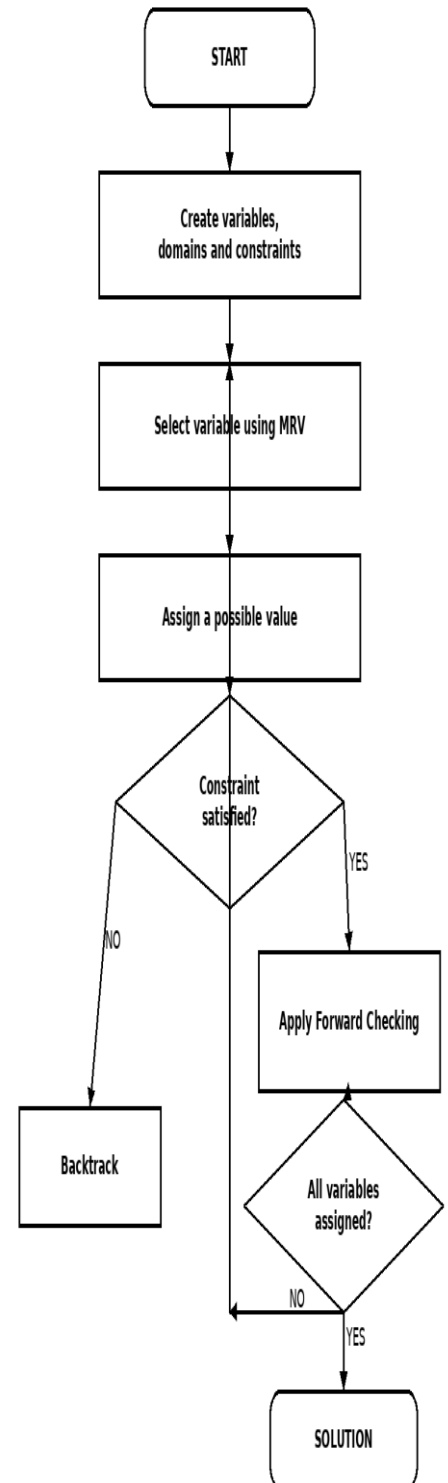
11. Conclusion

CSP with MRV and Forward Checking provides a systematic and efficient way to construct a conflict-free examination timetable.

Example constraints: $A \neq B$, $A \neq C$, $B \neq D$, $C \neq D$, $D \neq E$.

5. Flowchart / Algorithm Diagram

CSP Timetable Flowchart



QUESTION 4 – Minimax and Alpha-Beta Pruning

Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha-Beta Pruning and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.

1. Problem Statement

Minimax is used in a two-player game. MAX tries to maximize the score and MIN tries to minimize it.

2. Problem Analysis

- MAX node chooses the maximum value
- MIN node chooses the minimum value

3. AI Technique Selection

Minimax followed by Alpha-Beta Pruning.

4. Justification of Algorithm

Minimax determines the best move assuming an optimal opponent. Alpha-Beta gives the same final decision while skipping branches that cannot influence the result.

6. Algorithm / Pseudocode

Minimax(node, maximizingPlayer)

1. If node is terminal, return utility.
2. If MAX, return maximum child value.
3. If MIN, return minimum child value.

AlphaBeta(node, alpha, beta, player)

1. If terminal, return utility.
2. For MAX: update alpha after each child.
3. For MIN: update beta after each child.

7. Working / Execution Trace

Illustrative tree: $A = \text{MIN}(3, 5) = 3$; $B = \text{MIN}(2, 9) = 2$; $\text{root} = \text{MAX}(3, 2) = 3$. Best move for MAX is A. After A gives $\alpha = 3$, at B the first value 2 gives $\beta = 2$, so 9 is pruned because $\alpha \geq \beta$.

8. Complexity Analysis

Minimax worst case: $O(b^d)$. Alpha-Beta worst case: $O(b^d)$, with excellent ordering it can approach $O(b^{(d/2)})$.

9. Advantages

- Produces the best move against an optimal opponent; Alpha-Beta reduces unnecessary evaluations.

10. Limitations

- Large game trees can still be expensive; pruning effectiveness depends on move ordering.

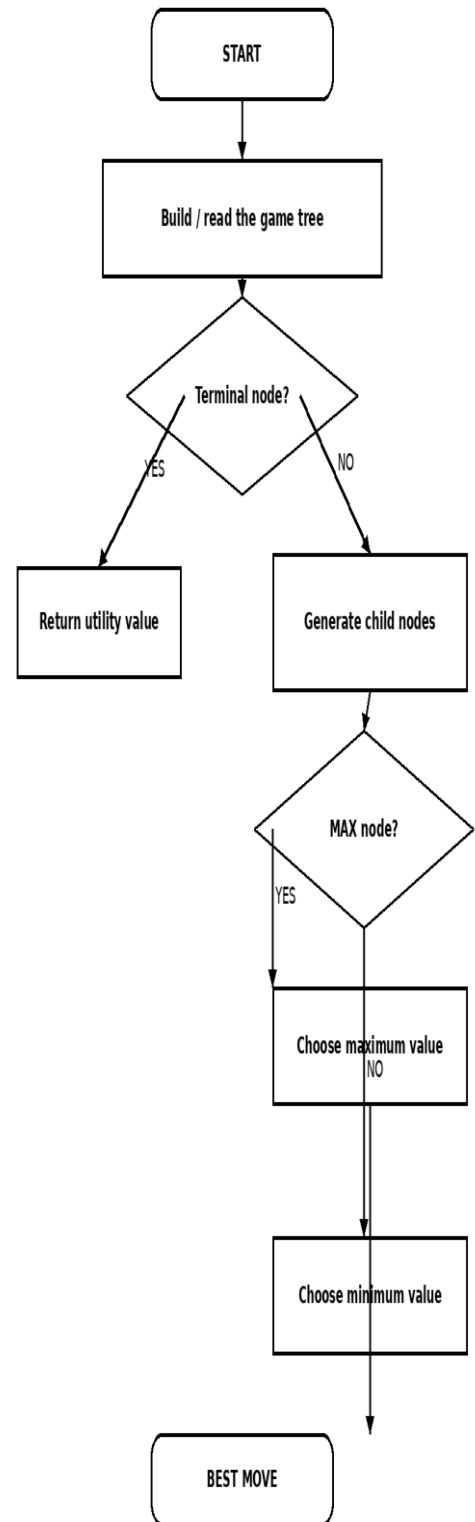
11. Conclusion

Minimax gives the best decision for MAX, while Alpha-Beta improves efficiency by pruning irrelevant branches.

Illustrative comparison: without pruning, all four leaves (3, 5, 2, 9) are evaluated; with Alpha-Beta, leaf 9 is skipped. The final choice remains A.

5. Flowchart / Algorithm Diagram

Minimax Flowchart



QUESTION 5 – Online Search Agent for a Warehouse Robot

A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts and compare it with informed search algorithms.

1. Problem Statement

Reach a warehouse delivery destination while obstacles may appear unexpectedly. An Online Search Agent is selected because it can observe changes and re-plan during execution.

2. Problem Analysis

- Initial state: Robot's current position
- Goal: Delivery location
- Actions: Move up, down, left, or right
- Environment: Dynamic

3. AI Technique Selection

Online Search Agent.

4. Justification of Algorithm

It is suitable because the complete environment may not be known in advance. The robot can observe the current state, move, detect obstacles, update its map, and re-plan.

6. Algorithm / Pseudocode

```
Online_Search_Robot(start, goal)
1. current = start.
2. While current != goal:
  a. Observe the environment.
  b. Detect obstacles.
  c. Generate possible actions.
  d. Select the best available action.
  e. Move the robot.
  f. If a new obstacle appears:.
3. Stop when goal is reached.
```

7. Working / Execution Trace

Example: If the original route is $S \rightarrow \rightarrow \rightarrow G$ and an obstacle X appears, the robot detects X , updates its map, chooses another route, and continues until it reaches G .

8. Complexity Analysis

Complexity depends on the number of states, available actions, environmental changes, and frequency of re-planning.

9. Advantages

- Adapts to unexpected obstacles; works with incomplete knowledge; re-plans during execution.

10. Limitations

- May produce a longer route and requires additional computation when the environment changes.

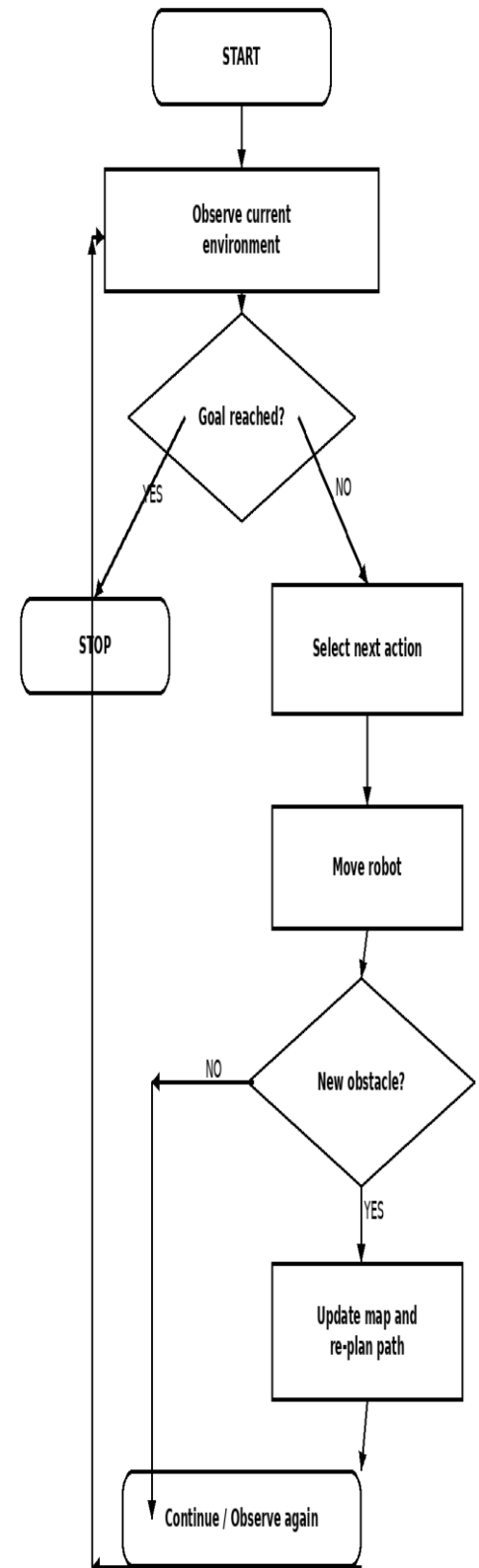
11. Conclusion

An Online Search Agent is appropriate for a dynamic warehouse because it can observe changes and re-plan while moving.

Comparison: Online Search is highly adaptable to changing environments, while informed search such as A^* normally relies on an available model.

5. Flowchart / Algorithm Diagram

Online Search Agent Flowchart



- Problem Analysis
- AI Technique Selection
- Justification
- Flowchart / Diagram
- Heuristic or CSP details where applicable
- Pseudocode
- Working / Execution Trace
- Complexity Analysis
- Advantages and Limitations
- Conclusion

The document is editable. You can change the illustrative Romania Map values and game-tree values after checking the exact graph/tree provided by your faculty.